}

Scenario 4:

```
# include <iostream>
# include <string>

using namespace std;

class WeekDays {
private:
    string Days[7];
    int currentDay;
```

CamScanner

Dated:

```
public:
    WeekDays() {
        days[0] = "Sunday";
        days[1] = "Monday";
        days[2] = "Tuesday";
        days[3] = "Wednesday";
        days[4] = "Thursday";
        days[5] = "Friday";
        days[6] = "Saturday";
        currentDay = 0;
    }
WeekDays(int dayIndex) {
        days[0] = "Sunday";
        days[1] = "Monday";
        days[2] = "Tuesday";
        days[3] = "Wednesday";
        days[4] = "Thursday";
        days[5] = "Friday";
        days[6] = "Saturday";

    currentDay = (dayIndex % 7 + 7) % 7;
    }
string getCurrentDay() const {
        return days[currentDay];
}

string getNextDay() const {
        return days[(currentDay + 1) % 7];
}
```

Dated:

```
string getPreviousDay() const {
    return days[(currentDay - 1 + 7) % 7];
}

string getNthDayFromToday(int n) const {
    int targetIndex = (currentDay + (n % 7) + 7) % 7;
    return days[targetIndex];
}
};

int main() {
    WeekDays defaultWeek;
    cout << "= Default Constructor Demonstration" << endl;
    cout << "Current Day: " << defaultWeek.getCurrentDay() << endl;
    cout << "Next Day: " << defaultWeek.getNextDay() << endl;
    cout << "Previous Day: " << defaultWeek.getPreviousDay() << endl;

    int inputDay;
    cout << "Enter an integer value for the current Day";
    cin >> inputDay;

    WeekDays userWeek(inputDay);
    cout << "Parameterized Constructor" << endl;
    cout << "Current Day: " << userWeek.getCurrentDay() << endl;
    cout << "Next Day: " << userWeek.getNextDay() << endl;
    cout << "Previous Day: " << userWeek.getPreviousDay() << endl;
    int nDays;
    cout << "Enter a number of Days to look forward";
    cin >> nDays;
    cout << "Day after " << nDays << " days: " << userWeek.getNthWeek
```

Dated: _____

from Today (nDays) << endl;

return 0;

}

